Quick tour of other Deep Bayes ideas

- What is *sampling* of models?
- How to sample?

Begin Training Dynamics, if time

Bayesian Model Averaging as graphical model marginalization

Derive on board

$$\mathbb{E}_{p(w|D)}[p(y|x,w)] = \int p(y|x,w)p(w|D)dw$$

Why we can *sample* to approximate expectations

$$\mathbb{E}_{p(w|D)}[p(y|x,w)] = \int p(y|x,w)p(w|D)dw$$

$$\approx \frac{1}{N}\sum_{i=1..N} p(y|x,w_i) \quad \text{with } w_i \sim p(w|D)$$

(Does it seem mysterious when we do this? Can try to understand on board…)

Bayesian sampling written as sampling an Energy-based model

$$p(w|D) = e^{-E(w)}/Z$$

With the energy defined:

$$E(w) = -\sum_i \log p(y_i|x_i, w) - \log p(w)$$

Likelihood

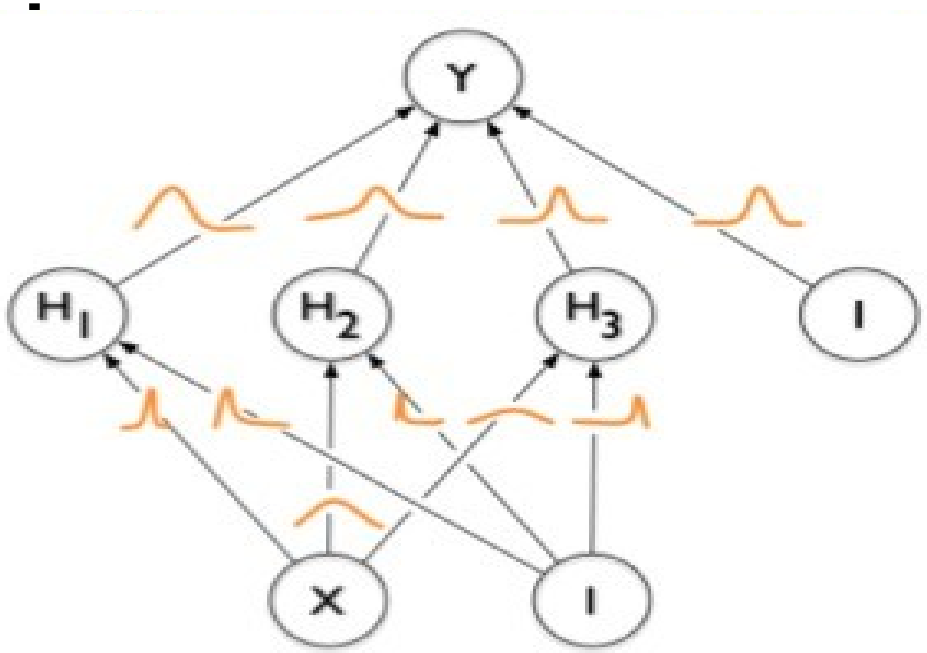
Prior

Could be Gaussian for neural nets, leading to |w|^2

Variational Bayes – approximate with simple distribu

$$q(\mathbf{w}) = \mathcal{N}(\mathbf{w}; \boldsymbol{\mu}, \boldsymbol{\Sigma})$$
$$= \prod_{j=1}^{D} \mathcal{N}(\theta_j; \mu_j, \sigma_j)$$

This means each weight of the network has its own mean and variance.



— Blundell et al., Weight uncertainty for neural networks

We can train a variational BNN using the same reparameterization trick as from VAEs.

$$\theta_j = \mu_j + \sigma_j \epsilon_j,$$

where $\epsilon_j \sim \mathcal{N}(0, 1)$.

More variational inference later in course

Bayesian sampling written as sampling an Energy-based model

$$p(w|D) = e^{-E(w)}/Z$$

With the energy defined:

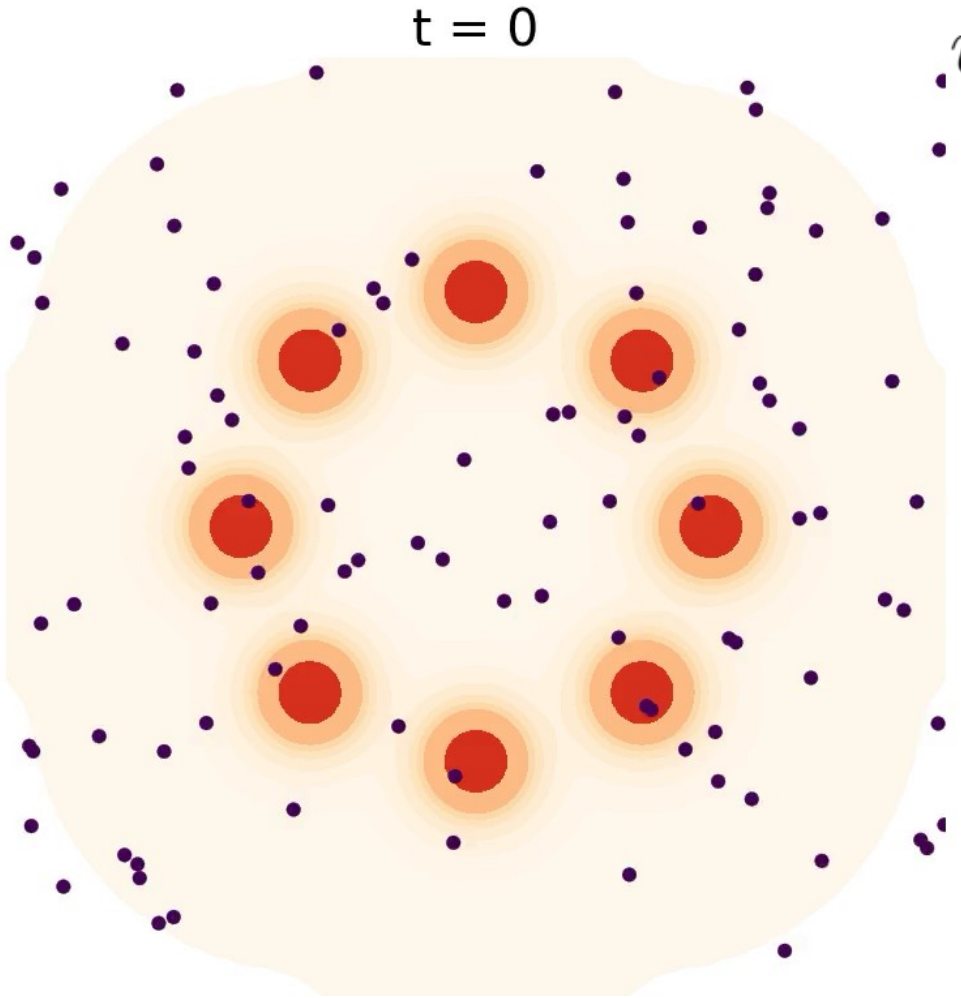
$$E(w) = -\sum_i \log p(y_i|x_i, w) - \log p(w)$$

Likelihood

Prior

Could be Gaussian for neural nets, leading to |w|^2

Langevin dynamics*



Weight space

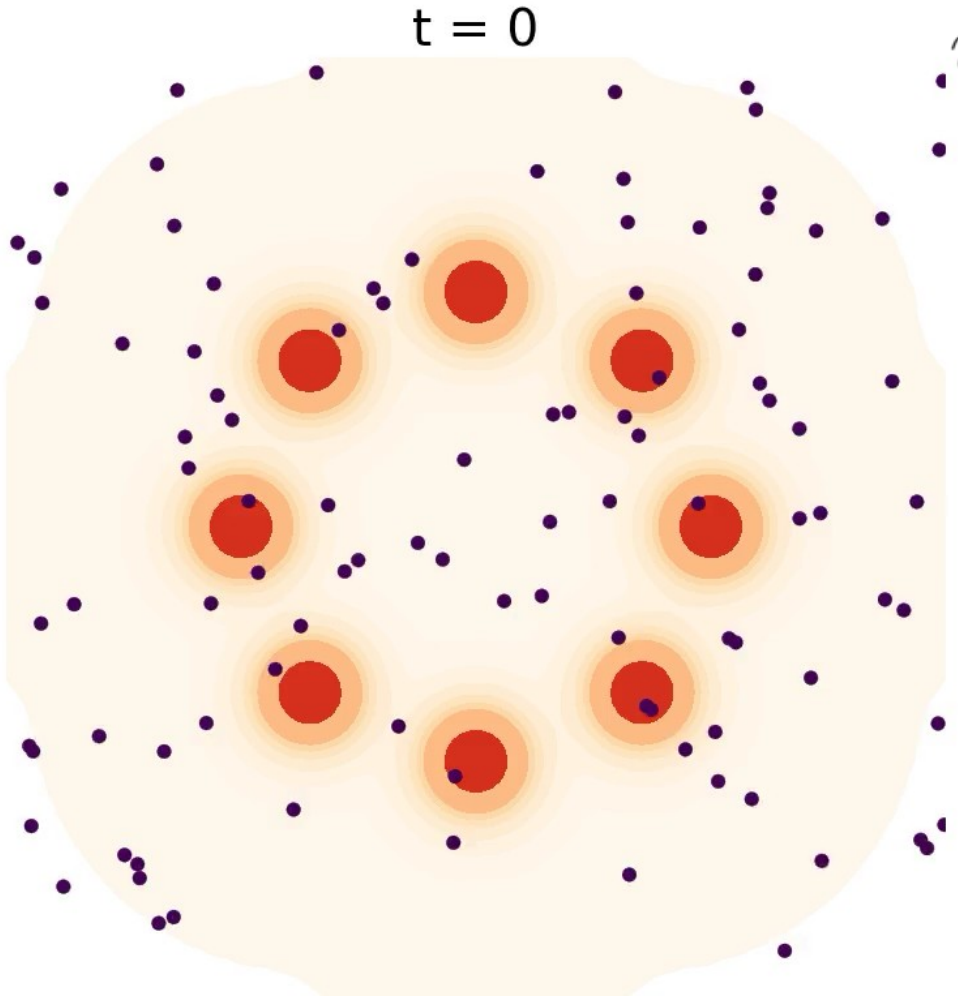
$$w_{t+\eta} = w_t - \eta \nabla_w E(w_t) + \sqrt{2\eta}\, v$$

Gradient step toward low energy / high probability

Noise to explore $v \sim \mathcal{N}(0, \mathbb{I})$

**Technically, must add Metropolis-Hastings rejection step to make it a valid sampler, or take step size to zero, which no one does*

Problem Solved? Not so fast



Weight space

$$w_{t+\eta} = w_t - \eta \nabla_w E(w_t) + \sqrt{2\eta}\, v$$

Require FULL BATCH backprop for each step!!!

$$E(w) = -\sum_i \log p(y_i|x_i, w) - \log p(w)$$

Sampling with mini-batches papers

- **Bayesian learning via stochastic gradient Langevin dynamics.** Welling, M., Teh, Y. ICML 2011
- Stochastic gradient Hamiltonian Monte Carlo. Chen, T., Fox, E., Guestin, C. ICML 2014
- Cyclical stochastic gradient MCMC for Bayesian deep learning. Zhang et. al, ICLR 2020

Welling & Teh, Stochastic Gradient Langevin Dynamics

$$w_{t+1} - w_t = -\eta\, g_{minibatch}$$

From central limit theorem, we say that:

$$g_{minibatch} = \frac{1}{b} \sum_{j \in batch} g_j$$

$$g_{minibatch} \sim N(\bar{g}, \frac{\Sigma}{b})$$

$$w_{t+1} - w_t = -\eta\, g_{minibatch} = -\eta\, (\bar{g} + \frac{\Sigma^{\frac{1}{2}}}{\sqrt{b}}\, \epsilon)$$

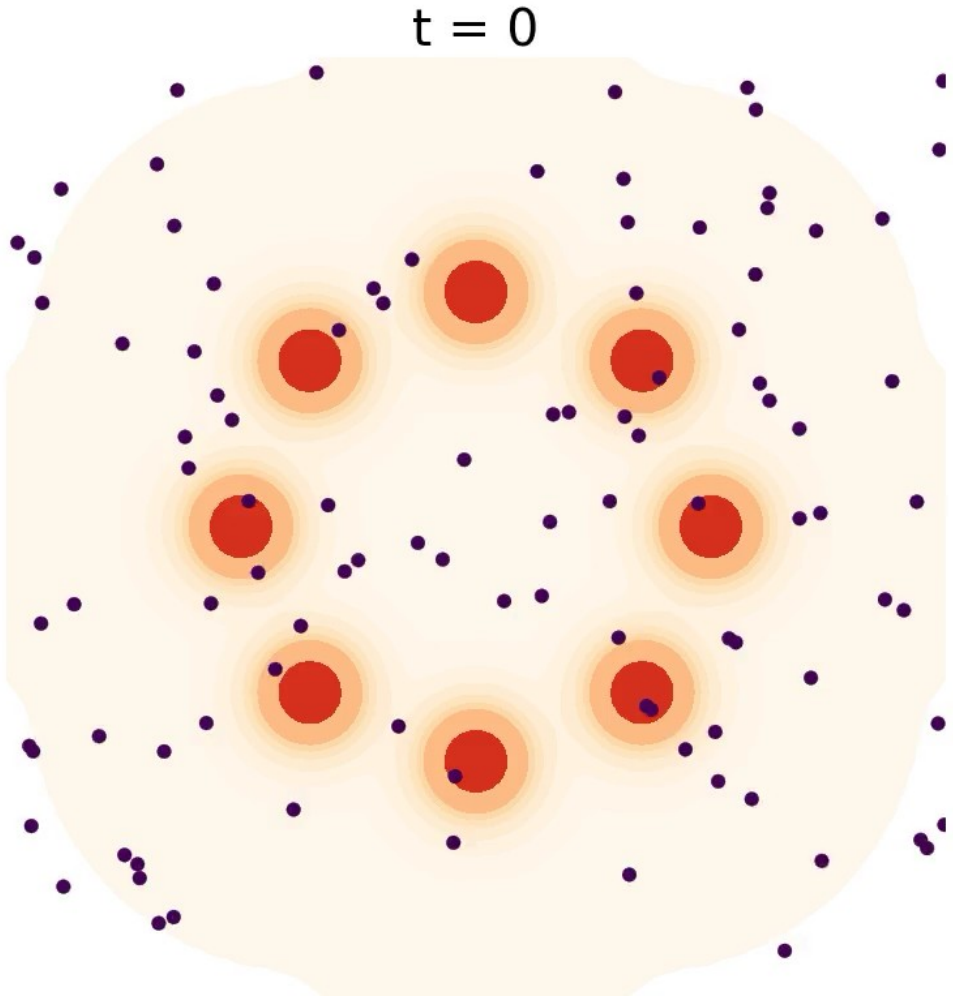
If we identify $\mathrm{g}(w) = \bar{g}, C = \sqrt{\frac{\eta}{b}\Sigma}$

$$w_{t+1} - w_t = -g(w_t)\eta + C(w_t)\sqrt{\eta}\,\epsilon$$

Problem

Let the step size, eta, go to zero, while adding a small amount of spherical Gaussian noise

Problem Solved?

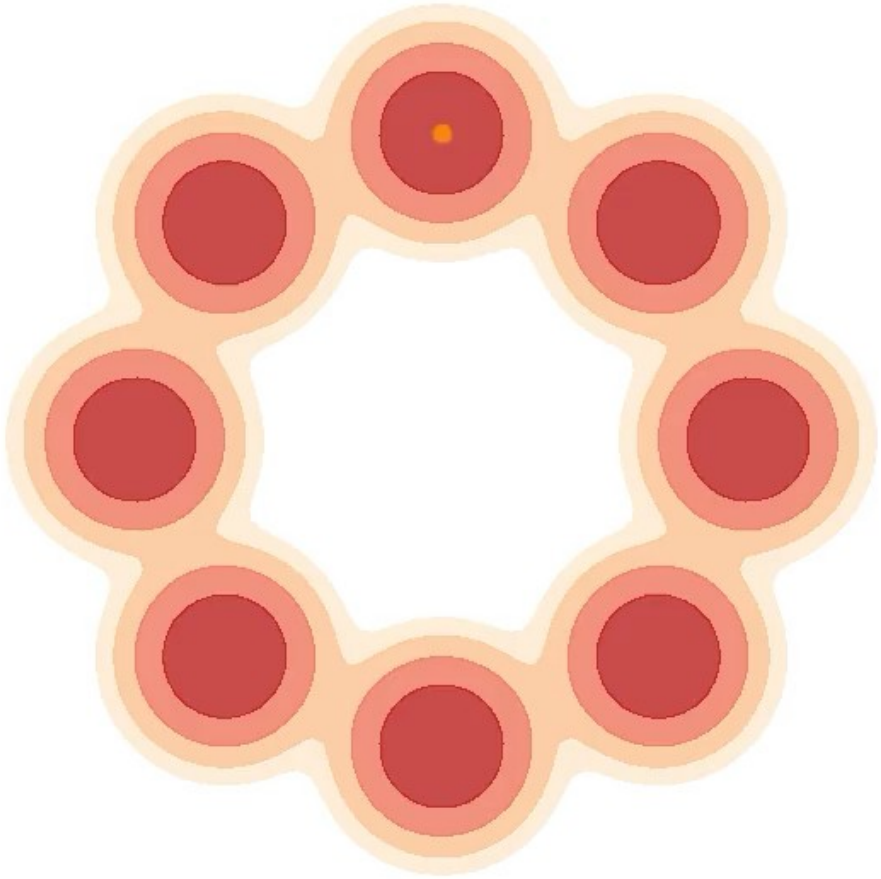


$$w_{t+\eta} = w_t - \eta \nabla_w E(w_t) + \sqrt{2\eta}\, v$$

Use small learning rate, stochastic gradient, and add some gaussian noise

$$E(w) = -\sum_i \log p(y_i|x_i, w) - \log p(w)$$

Interesting side note:
if the dynamics sample weights over time, maybe instead of multiple samples we could look at one trajectory over time



Sampling one trajectory, over time

Langevin dynamics

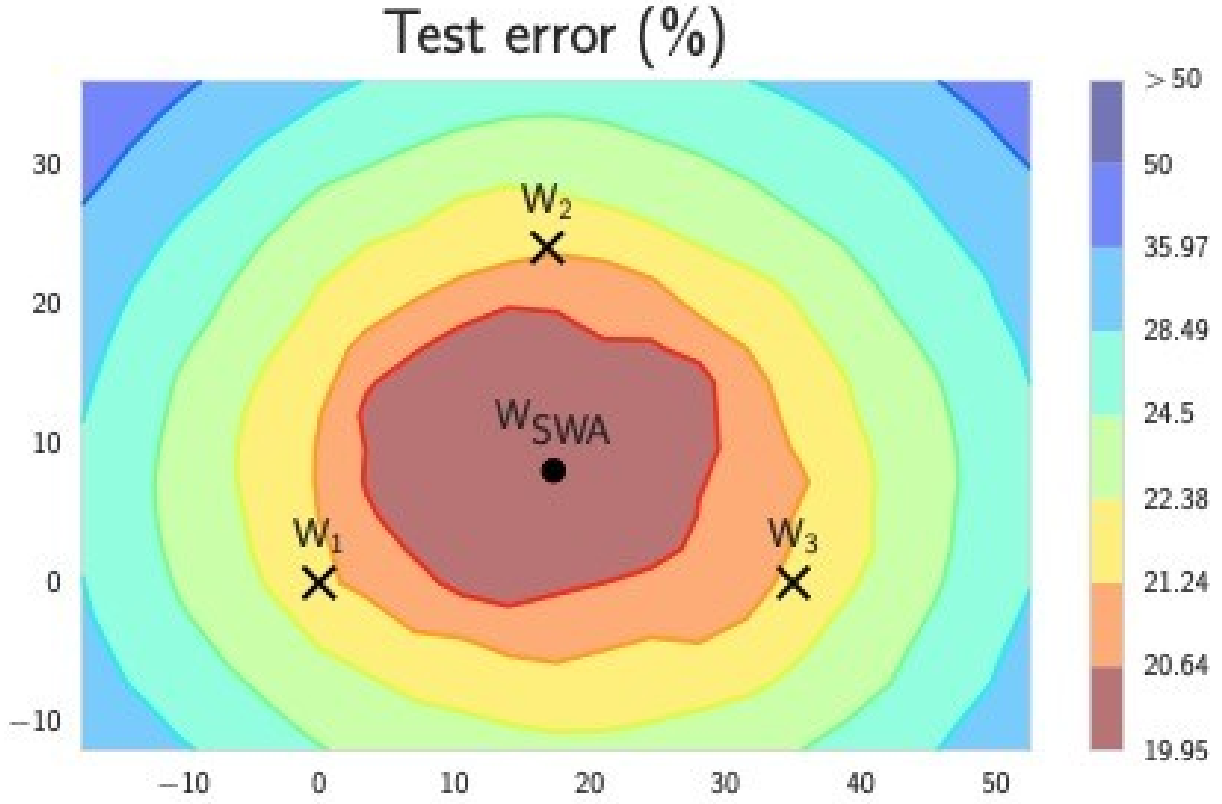
Micro-canonical sampling dynamics

(from my 2021 Neurips paper on “Non-Newtonian Momentum..”)

Width corresponds to time spent in each region (in unscaled time)

(Great follow-up work by Robnik et al)

Not Bayesian – but averaging samples over time (Stochastic Weight Averaging) really seems to work!



Lots of weight averaging approaches
Federated learning, Polyak averaging

Fast ensembling (Stochastic Weight Averaging)

Algorithm 1 Stochastic Weight Averaging

Require:
weights $\hat{w}$, LR bounds α_1, α_2,
cycle length c (for constant learning rate $c = 1$), number of iterations n

Ensure: w_{SWA}
$w \leftarrow \hat{w}$ {Initialize weights with $\hat{w}$}
$w_{\text{SWA}} \leftarrow w$
for $i \leftarrow 1, 2, \ldots, n$ **do**
  $\alpha \leftarrow \alpha(i)$ {Calculate LR for the iteration}
  $w \leftarrow w - \alpha \nabla \mathcal{L}_i(w)$ {Stochastic gradient update}
  if $\text{mod}(i, c) = 0$ **then**
    $n_{\text{models}} \leftarrow i/c$ {Number of models}
    $w_{\text{SWA}} \leftarrow \frac{w_{\text{SWA}} \cdot n_{\text{models}} + w}{n_{\text{models}} + 1}$ {Update average}
  end if
end for
{Compute BatchNorm statistics for w_{SWA} weights}

Regular SGD update

Track the moving average

Exponential Moving Average of model weights

In practice, simpler, more common.

Update a moving average on each step.

$$w_{EMA} \leftarrow \beta w_{EMA} + (1 - \beta)\theta_{model}$$

$$\beta = 0.99 \ - 0.9999$$

SWAG

A Simple Baseline for Bayesian Uncertainty in Deep Learning

Algorithm 1 Bayesian Model Averaging with SWAG

θ_0: pretrained weights; η: learning rate; T: number of steps; c: moment update frequency; K: maximum number of columns in deviation matrix; S: number of samples in Bayesian model averaging

Train SWAG

$\overline{\theta} \leftarrow \theta_0, \;\; \overline{\theta^2} \leftarrow \theta_0^2$ {Initialize moments}

for $i \leftarrow 1, 2, ..., T$ **do**

$\theta_i \leftarrow \theta_{i-1} - \eta \nabla_\theta \mathcal{L}(\theta_{i-1})$ {Perform SGD update}

if $\texttt{MOD}(i, c) = 0$ **then**

$n \leftarrow i/c$ {Number of models}

$\overline{\theta} \leftarrow \frac{n\overline{\theta} + \theta_i}{n+1}, \;\; \overline{\theta^2} \leftarrow \frac{n\overline{\theta^2} + \theta_i^2}{n+1}$ {Moments}

if $\texttt{NUM\_COLS}(\widehat{D}) = K$ **then**

$\texttt{REMOVE\_COL}(\widehat{D}[:, 1])$

$\texttt{APPEND\_COL}(\widehat{D}, \theta_i - \overline{\theta})$ {Store deviation}

return $\theta_{\text{SWA}} = \overline{\theta}, \;\; \Sigma_{\text{diag}} = \overline{\theta^2} - \overline{\theta}^2, \;\; \widehat{D}$

Test Bayesian Model Averaging

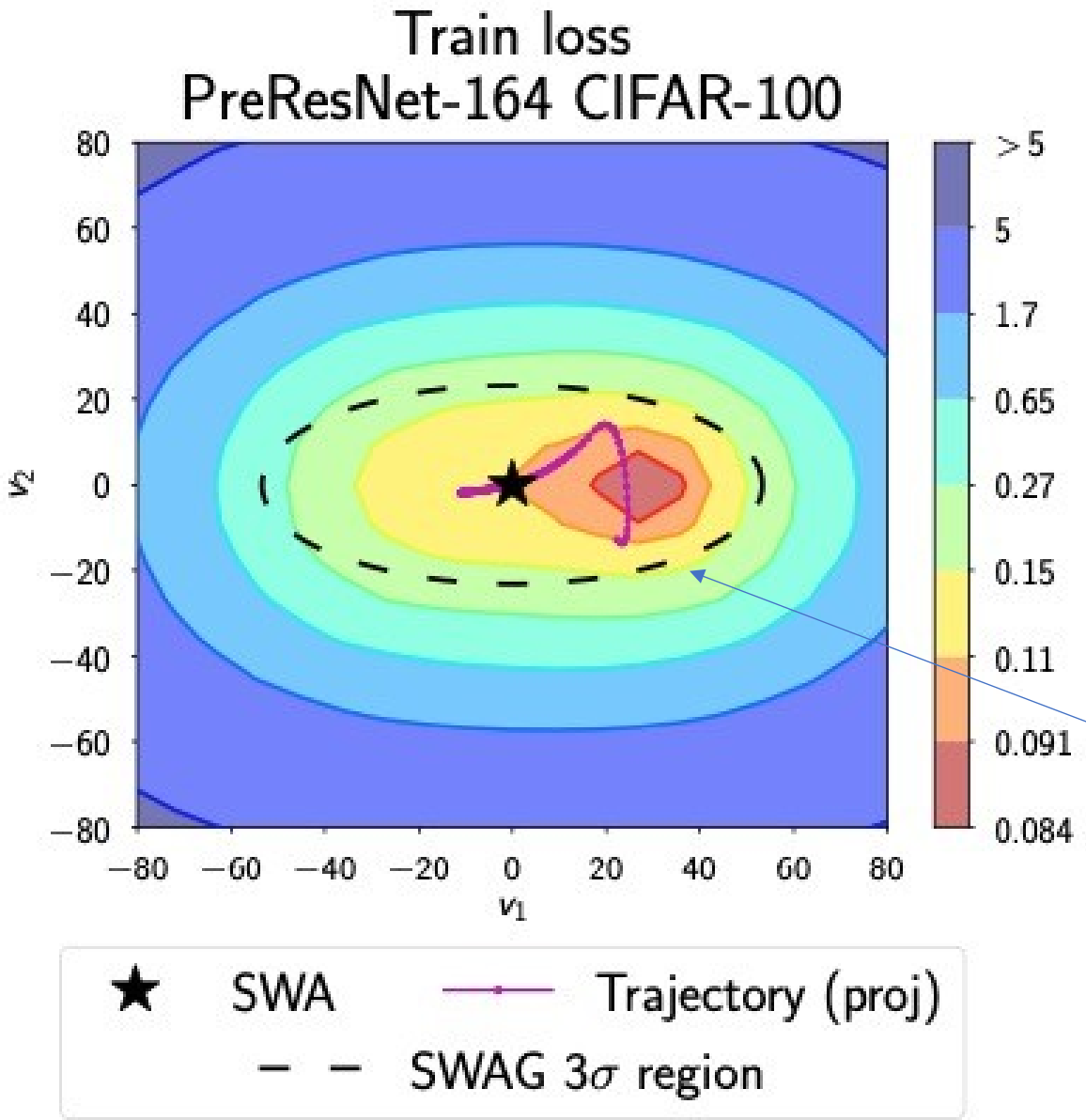
for $i \leftarrow 1, 2, ..., S$ **do**

Draw $\widetilde{\theta}_i \sim \mathcal{N}\left(\theta_{\text{SWA}}, \frac{1}{2}\Sigma_{\text{diag}} + \frac{\widehat{D}\widehat{D}^\top}{2(K-1)}\right)$ (1)

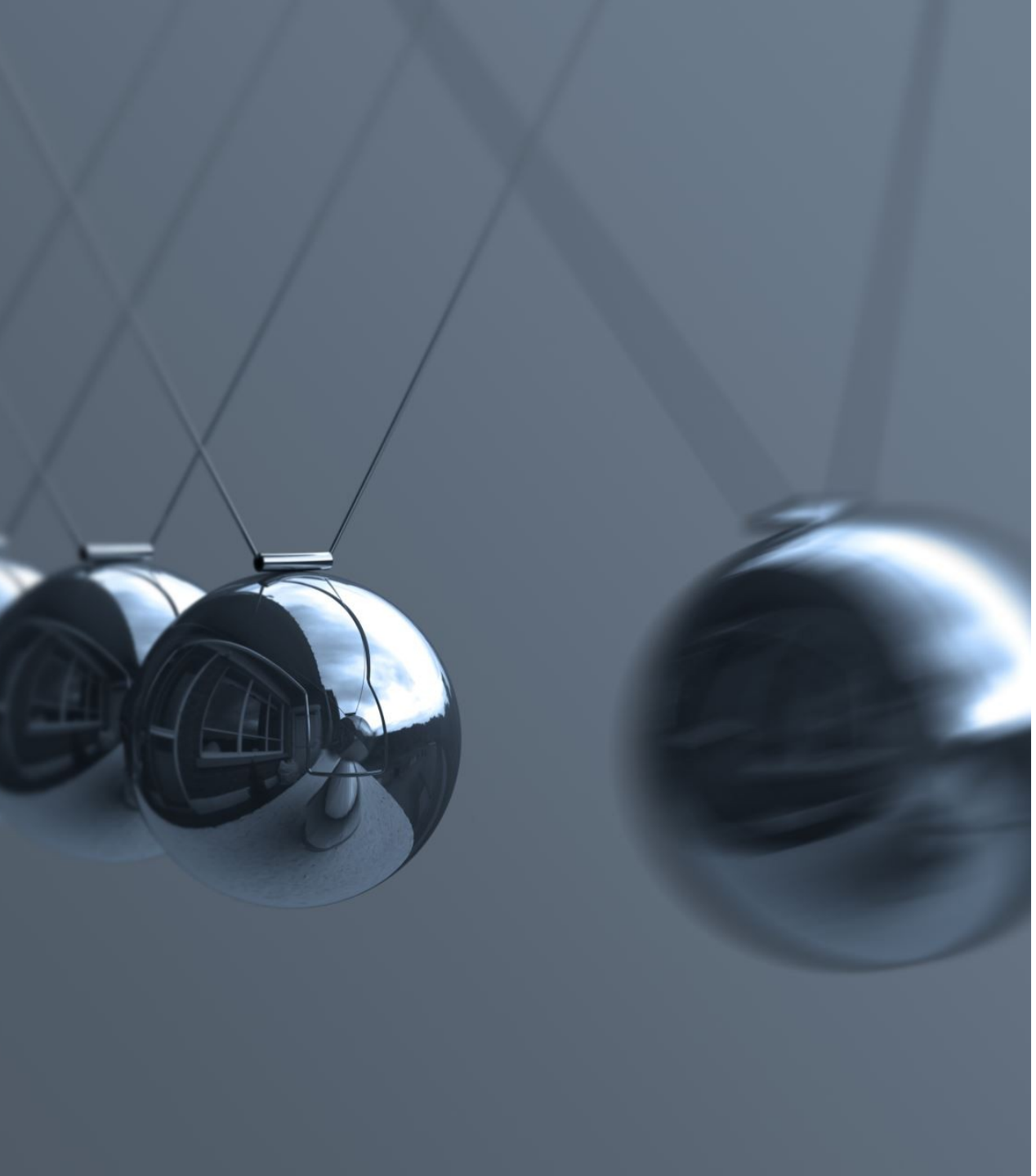
Update batch norm statistics with new sample.

$p(y^*|\text{Data}) \mathrel{+}= \frac{1}{S} p(y^*|\widetilde{\theta}_i)$

return $p(y^*|\text{Data})$



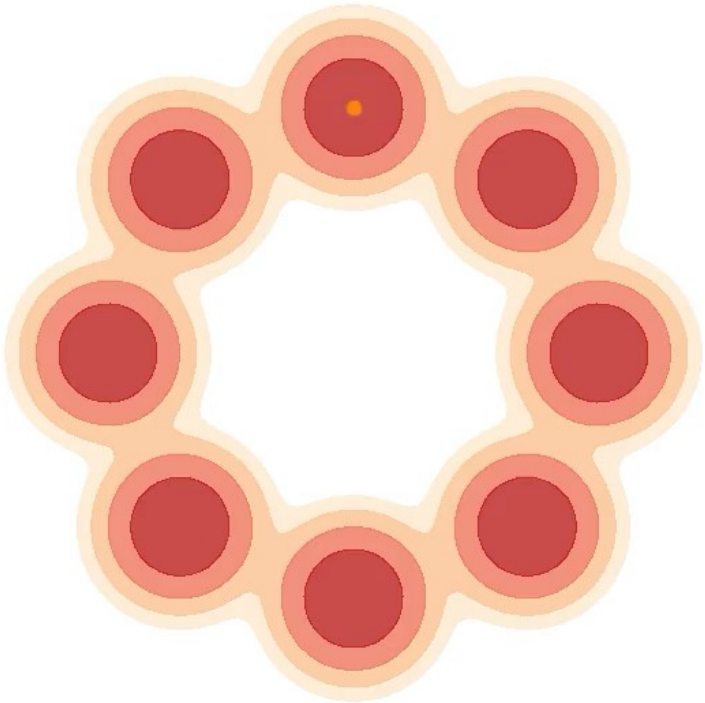
Sample solutions from this learned approximate Gaussian, for Bayesian model averaging



Importance of dynamics in ML

Sampling uses dynamics

- Contour plot represents high probability regions
- Dynamics samples high probability states over time
- Orange dot is Langevin dynamics, stochastic dynamics
- Blue dot is deterministic dynamics

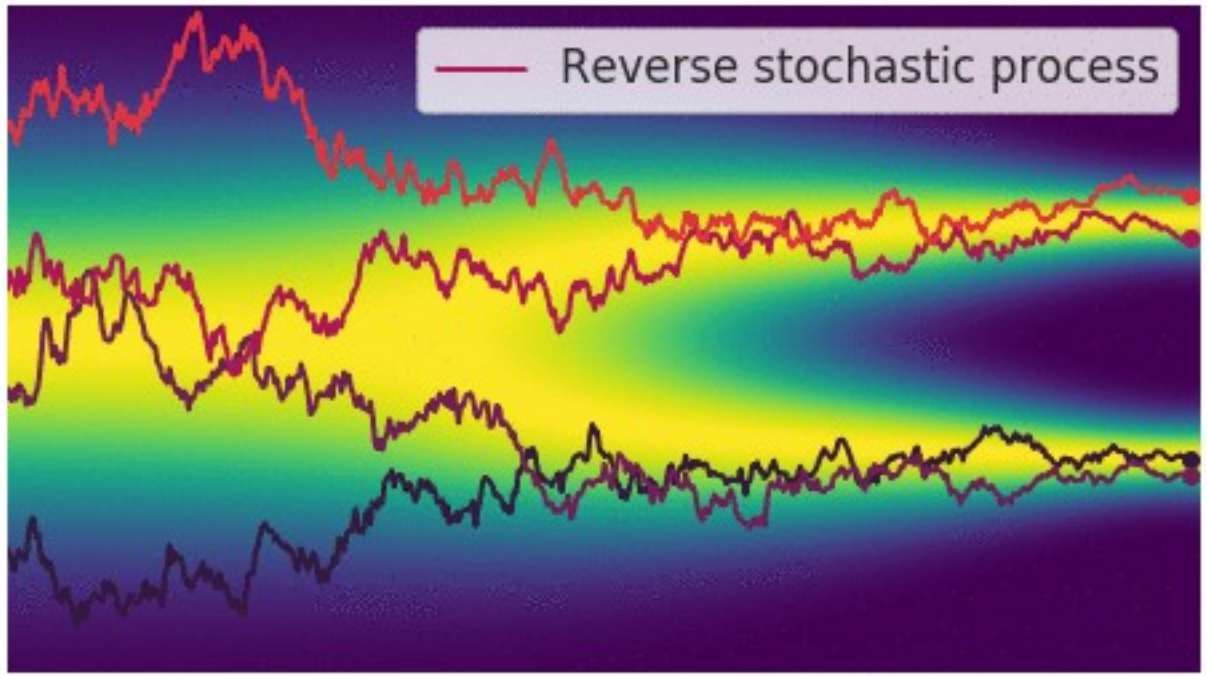


Dynamics and local optima

Loss landscape of a ResNet-20

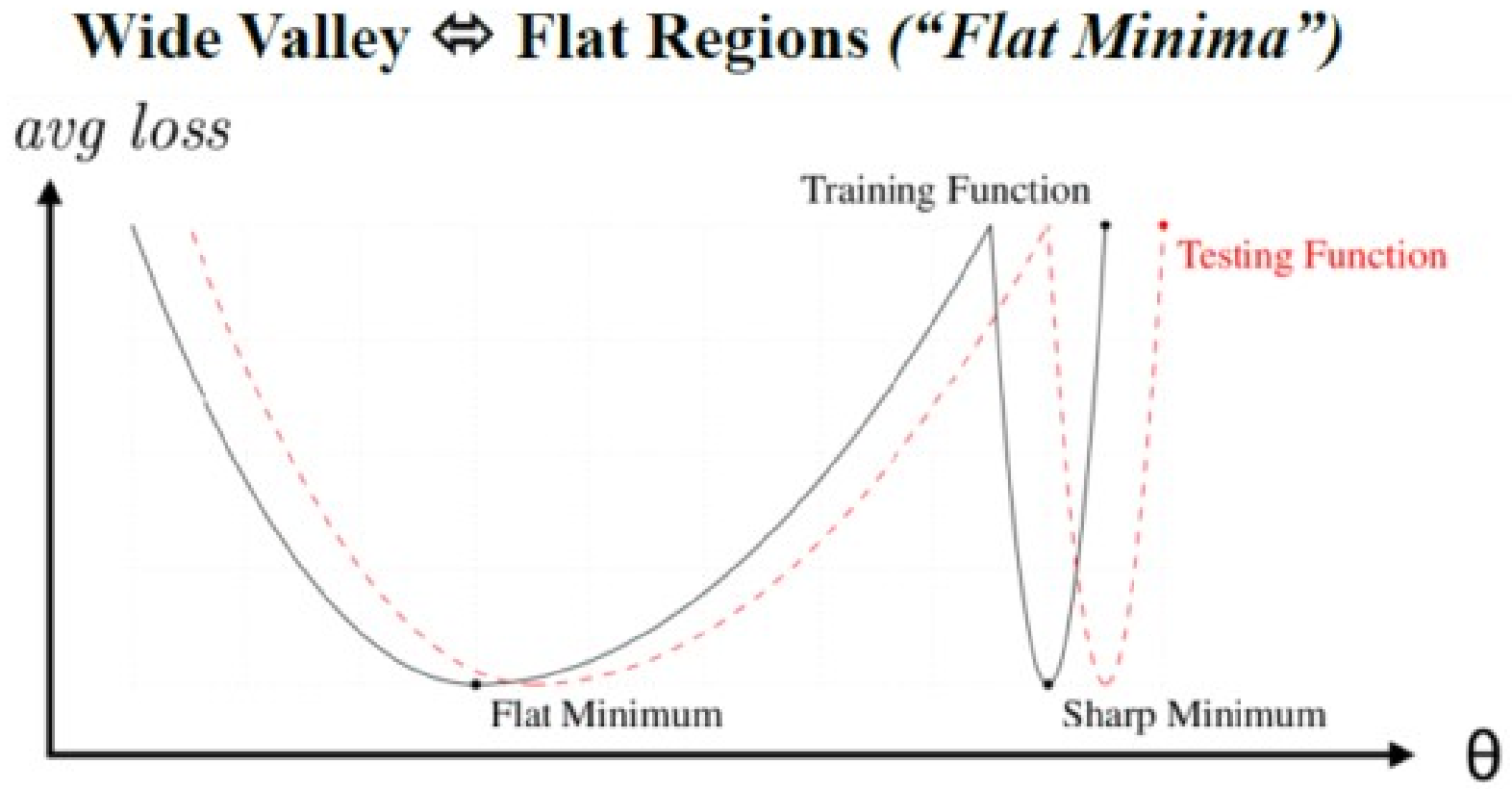
https://izmailovpavel.github.io/curves_blogpost

Generative models as dynamics



https://yang-song.github.io/blog/2021/score/

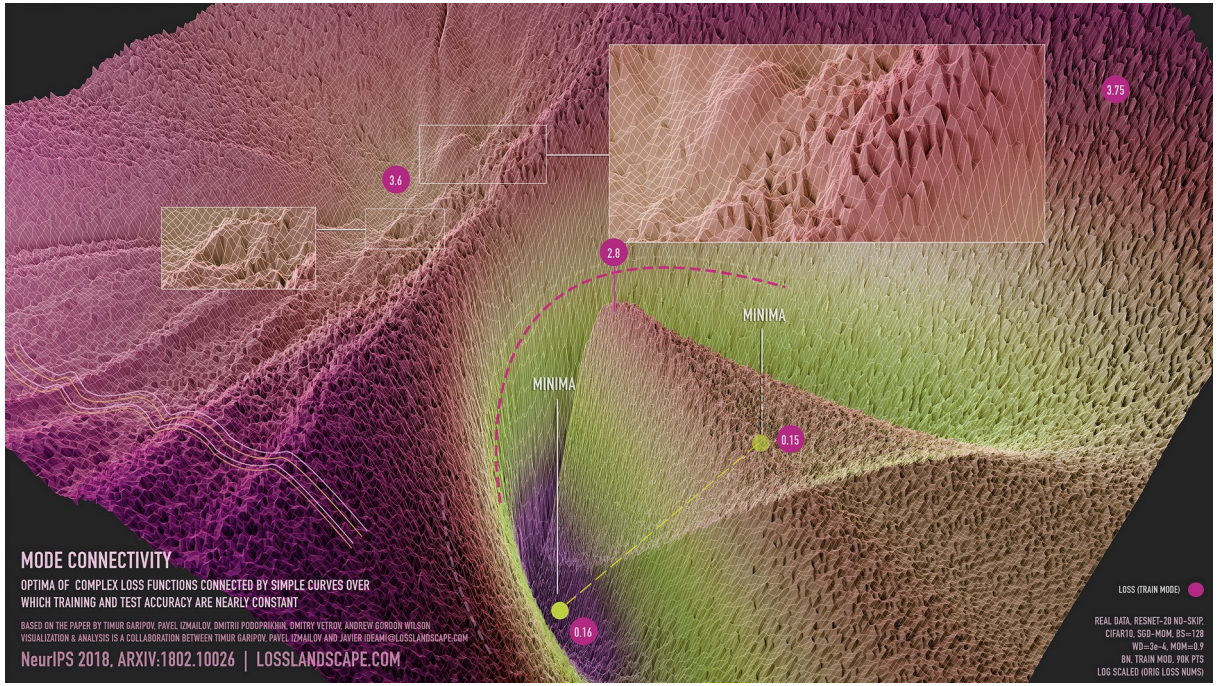
Why deep learning works? (Good) optimization dynamics prefer generalizable solutions (?)



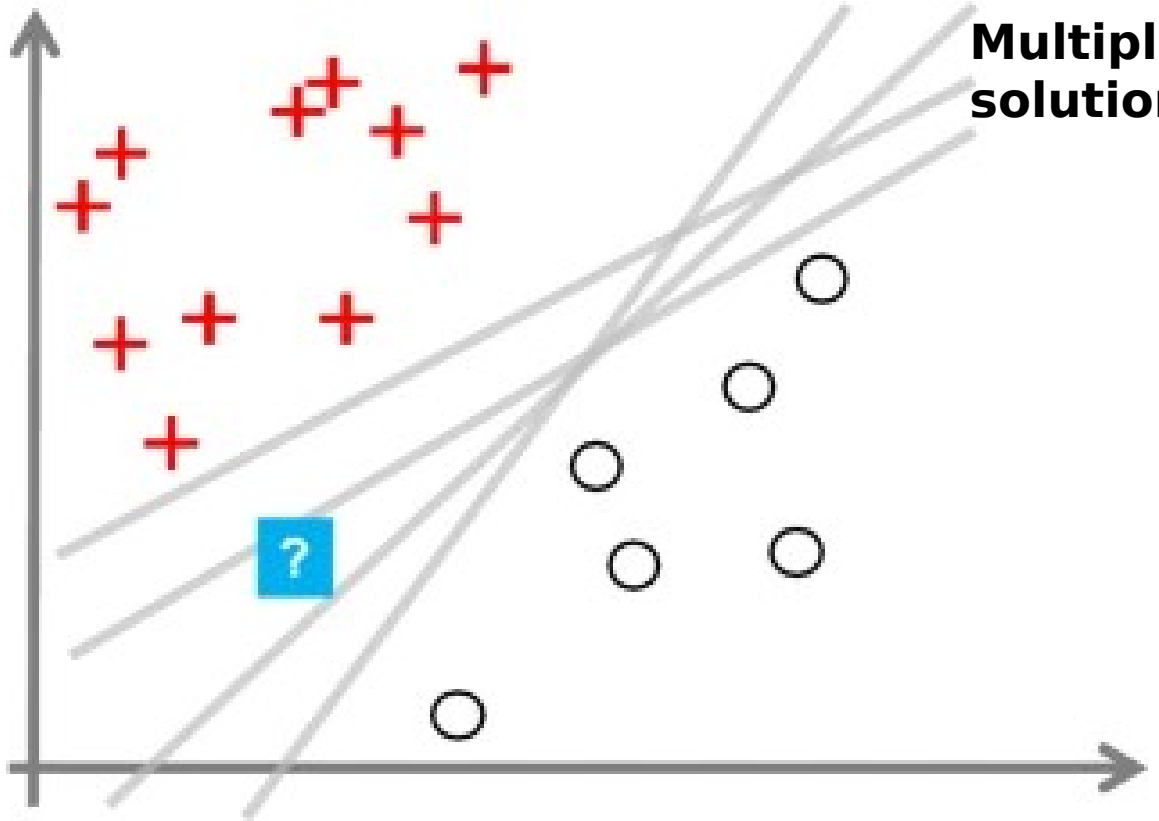
Keskar et al. (2017) "On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima". CoRR

Optimization versus sampling (with dynamics) – when/why we want multiple solutions

Loss landscape – many good solutions



Classifier



If we only look at one solution we may be confidently wrong with our predictions

Get into training dynamics details next week!

- Momentum
- Adam vs SGD
- Why SGD works